Moringa School — Generative AI Capstone

Getting Started with Automated Data Quality Auditing in Python

A Beginner-Friendly Toolkit for Detecting & Flagging Dirty Data

Author	Judith
Technology	Python · pandas · numpy · matplotlib · seaborn
Platform	Jupyter Notebook
AI Tool	Claude
Dataset	dirty_ecommerce_dataset.csv

1 🎯 Title & Objective

Chosen Technology: Python — automated data quality auditing using the pandas and numpy libraries inside a Jupyter Notebook.

Why this technology?

- pandas is already a core tool in the data science workflow — this project extends existing knowledge rather than starting from scratch.
- Data quality problems are pervasive. Every dataset used for analysis, reporting, or machine learning must pass a quality gate first.
- Automation is the key skill gap: most junior analysts check data manually, row by row. This toolkit replaces that with a systematic, reusable audit.
- The output (boolean FLAG_ columns + quality score) is immediately actionable and easy to hand off to a data cleaning step.

End Goal

A Jupyter Notebook that accepts any CSV, scans it for nine categories of data issues, appends a boolean FLAG_ column per issue type, visualises the results, and prints an overall data quality score — all without manually inspecting a single row.

2 📖 Quick Summary of the Technology

What is it?

pandas is an open-source Python library providing fast, flexible, and expressive data structures, primarily the DataFrame, designed to make working with tabular data both easy and intuitive. It is the de-facto standard for data manipulation in Python. numpy underpins pandas with efficient array operations and mathematical functions.

Where is it used?

- Data science and machine learning pipelines (feature engineering, data cleaning)
- Business intelligence and financial reporting
- Healthcare analytics (patient record validation)
- E-commerce analytics (sales, inventory, customer data)
- Government and NGO data processing

One Real-World Example

> *A retail company receives a daily CSV export from its Point-of-Sale system. Before loading that file into their Power BI dashboard, an automated pandas audit runs every morning. It flags 34 rows where the revenue column is null, 12 duplicate transaction IDs, and 8 dates that failed to parse , all before a single analyst opens the file. Issues are caught, logged, and routed for correction without corrupting the dashboard numbers.*

3 System Requirements

Item	Requirement
Operating System	Windows 10/11, macOS 11+, or Linux (Ubuntu 20+)
Python version	3.8 or higher
Editor	Jupyter Notebook or JupyterLab
Core packages	pandas, numpy, matplotlib, seaborn, jupyter
Dataset	dirty_ecommerce_dataset.csv (CSV format)
Disk space	~500 MB for Python + packages

> ⚠ *Windows users: Always prefix file paths with r"..." (raw string) to prevent Python from misreading backslashes as Unicode escape sequences.*

4 Installation & Setup Instructions

Step 1 — Install Python

Download the latest Python 3 installer from python.org/downloads and run it. On Windows, tick "Add Python to PATH" before clicking Install.

Step 2 — Install required packages

Open Command Prompt (Windows) or Terminal (Mac/Linux) and run:

```
pip install pandas numpy matplotlib seaborn jupyter
```

Step 3 — Launch Jupyter Notebook

```
jupyter notebook
```

Your default browser will open the Jupyter file browser. Click New → Python 3 to create a new notebook.

Step 4 — Place your dataset and update the path

Confirm your CSV file is in your Downloads folder, then update Cell 1 of the notebook:

```
# Windows (use r-string to avoid path errors)
df = pd.read_csv(r"C:\Users\JUDITH\Downloads\dirty_ecommerce_dataset.csv")

# Mac / Linux
df = pd.read_csv("~/Downloads/dirty_ecommerce_dataset.csv")
```

5 Minimal Working Example

The notebook scans the e-commerce CSV and adds a boolean FLAG_ column for every data quality issue found. At the end it computes an overall quality score: the percentage of rows that have zero flagged issues. A flagged copy of the dataset is saved to outputs/.

Cell 1 — Imports & load data

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from IPython.display import display

# r-string required on Windows to handle backslashes safely
df = pd.read_csv(r"C:\Users\JUDITH\Downloads\dirty_ecommerce_dataset.csv")

print("Shape:", df.shape)
print("\nColumn dtypes:\n", df.dtypes)
display(df.head())
```

Expected output: Table of first 5 rows + column names and their data types.

Cell 2 — Flag missing values

```
print("\n=== MISSING VALUES ===")
missing     = df.isnull().sum()
missing_pct = (missing / len(df) * 100).round(2)
missing_report = pd.DataFrame({
    "missing_count": missing, "missing_%": missing_pct
}).query("missing_count > 0").sort_values("missing_%", ascending=False)
display(missing_report)

# Heatmap — yellow cells = missing values
plt.figure(figsize=(12, 4))
sns.heatmap(df.isnull(), cbar=False, yticklabels=False, cmap="YlOrRd")
plt.title("Missing value map"); plt.tight_layout(); plt.show()

for col in missing_report.index:
    df[f"FLAG_missing_{col}"] = df[col].isnull()
```

Cell 3 — Flag duplicates

```
print("\n=== DUPLICATES ===")
n_dupes = df.duplicated().sum()
print(f"Full duplicate rows: {n_dupes} ({n_dupes/len(df)*100:.2f}%)")
df["FLAG_duplicate_row"] = df.duplicated(keep=False)

id_cols = ["order_id", "customer_id"]   # adjust to your dataset
for col in id_cols:
    if col in df.columns:
        duped = df[col].duplicated(keep=False)
        df[f"FLAG_duplicate_{col}"] = duped
        print(f"  Duplicate {col}: {duped.sum()} rows")
```

Cell 4 — Flag type inconsistencies & bad dates

```
print("\n=== TYPE CHECKS ===")
for col in df.select_dtypes(include="object").columns:
    coerced = pd.to_numeric(df[col], errors="coerce")
    n_convertible = coerced.notna().sum()
    if n_convertible > 0 and n_convertible < len(df):
        df[f"FLAG_mixed_type_{col}"] = coerced.isna() & df[col].notna()
        print(f"  '{col}': {n_convertible} numeric values mixed with strings")

date_cols = ["order_date", "ship_date"]   # adjust to your dataset
for col in date_cols:
    if col in df.columns:
        parsed    = pd.to_datetime(df[col], errors="coerce")
        bad_dates = parsed.isna() & df[col].notna()
        df[f"FLAG_invalid_date_{col}"] = bad_dates
        print(f"  '{col}': {bad_dates.sum()} unparseable dates")
```

Cell 5 — Flag outliers (IQR method)

```
print("\n=== OUTLIERS (IQR method) ===")
numeric_cols = df.select_dtypes(include=[np.number]).columns
numeric_cols = [c for c in numeric_cols if not c.startswith("FLAG_")]

for col in numeric_cols:
    Q1, Q3 = df[col].quantile(0.25), df[col].quantile(0.75)
    IQR    = Q3 - Q1
    lower, upper = Q1 - 1.5*IQR, Q3 + 1.5*IQR
    outliers = (df[col] < lower) | (df[col] > upper)
    df[f"FLAG_outlier_{col}"] = outliers
    if outliers.sum() > 0:
        print(f"  '{col}': {outliers.sum()} outliers (range {lower:.2f}-{upper:.2f})")

if "price"    in df.columns: df["FLAG_negative_price"] = df["price"] < 0
if "quantity" in df.columns: df["FLAG_negative_qty"]   = df["quantity"] < 0
```

Cell 6 — Flag string inconsistencies

```
print("\n=== STRING ISSUES ===")
for col in df.select_dtypes(include="object").columns:
    if col.startswith("FLAG_"):
        continue
    has_whitespace = df[col].str.match(r'^\s+|\s+$', na=False)
    unique_lower   = df[col].dropna().str.lower().nunique()
    unique_raw     = df[col].dropna().nunique()
    case_issue     = unique_raw > unique_lower
    df[f"FLAG_whitespace_{col}"] = has_whitespace
    if has_whitespace.sum() > 0:
        print(f"  '{col}': {has_whitespace.sum()} values with whitespace")
    if case_issue:
        print(f"  '{col}': casing inconsistency ({unique_raw} raw vs {unique_lower}
lowered)")
```

Cell 7 — Summary report & quality score

```
flag_cols = [c for c in df.columns if c.startswith("FLAG_")]
summary = []
for col in flag_cols:
    n = df[col].sum()
    summary.append({"flag": col, "rows_flagged": n,
                    "pct_of_dataset": round(n/len(df)*100, 2)})
summary_df = pd.DataFrame(summary).sort_values("rows_flagged", ascending=False)
display(summary_df)
```

```
any_issue     = df[flag_cols].any(axis=1)
quality_score = round((1 - any_issue.mean()) * 100, 1)
print(f"\nOverall quality score: {quality_score}% rows have no flagged issues")

plt.figure(figsize=(12, 6))
summary_df.set_index("flag")["rows_flagged"].plot(kind="barh", color="tomato")
plt.xlabel("Number of rows flagged")
plt.title("Data quality issues by flag type")
plt.tight_layout(); plt.show()

df.to_csv(r"C:\Users\JUDITH\Downloads\dirty_ecommerce_flagged.csv", index=False)
print("Flagged dataset saved.")
```

Expected Output

- A yellow-red heatmap where yellow cells indicate missing values
- Console output listing each flagged issue and the row count affected
- A horizontal bar chart ranking flag types by rows caught
- A printed quality score, e.g.: Overall quality score: 63.4% rows have no flagged issues
- A saved CSV at your Downloads folder with all FLAG_ columns appended

6 AI Prompt Journal

All prompts were submitted to Claude via ai.moringaschool.com. The goal was to use AI as a learning partner — explaining errors, suggesting structure, and deepening understanding — not as a code generator.

Prompt 1	Day: Monday
Prompt used:	"I've been programming in Python for 2 years, focusing on data analysis with pandas and numpy. Now I need to learn how to automate with Python. Could you give me a step-by-step process of writing Python code that flags discrepancies such as missing values, duplicates, inconsistencies in datasets? The dataset is a CSV in my Downloads folder and I want to write this in Jupyter Notebook."
AI response summary:	The AI produced a complete 7-cell notebook scaffold covering all major dirt categories, missing values, duplicates, type issues, outliers, and string inconsistencies, each in its own cell with inline comments and expected output described.
Helpfulness:	★★★★★ Excellent. Saved several hours of research and planning. The cell-by-cell structure was immediately usable and well-explained.

Prompt 2	Day: Monday
Prompt used:	"I'm getting a SyntaxError: unicode error on my file path — C:\Users\JUDITH\Downloads\dirty_ecommerce_dataset.csv"
AI response summary:	The AI immediately identified that backslashes in Windows paths are treated as Python Unicode escape sequences (\U, \D). It offered three fixes — raw string r"...", forward slashes, or double backslashes — and explained why each works.
Helpfulness:	★★★★★ Instant, precise diagnosis. I now use raw strings for all Windows paths — retained knowledge, not just a copy-paste fix.

Prompt 3	Day: Monday
Prompt used:	"My outlier and type-check code has a SyntaxError — [pasted the full broken 40-line cell]"
AI response summary:	The AI identified three distinct bugs in one response: (1) a stray opening parenthesis before a comment, (2) a space inside the column name string FLAG_negative_ qty, and (3) a double closing)) on the final print statement. Each bug was explained at the character level.
Helpfulness:	★★★★★ Surgical precision. No guessing — every fix was exact. Key lesson: paste the full broken code, not a description of it.

Prompt 4	Day: Monday
Prompt used:	"Is Step 6 (string inconsistencies check) necessary for an e-commerce dataset?"
AI response summary:	The AI explained that whitespace and casing issues are silent errors that don't surface as nulls but corrupt groupby and merge operations e.g. 'Electronics' and 'electronics ' are treated as two separate categories in a groupby, silently splitting revenue figures.
Helpfulness:	★★★★★ Deepened understanding beyond just making code run. Asking 'why' instead of 'how' is the most valuable AI prompt pattern.

Learning Reflection

> *Using AI as a learning partner rather than a code-generator was the defining insight of this project. The most productive prompts were specific: they included the exact error message, the exact broken code, and the exact question. Vague prompts like 'fix my code' produced generic answers; specific prompts with full context produced surgical fixes. Asking 'why is Step 6 necessary?' rather than just running the code built real understanding, the kind that transfers to future projects. Every time the AI explained a fix, it was read and understood before being applied, not copy-pasted blindly. That discipline is what turned AI assistance into genuine learning.*

7 🐛 Common Issues & Fixes

Error 1: SyntaxError — unicode escape on Windows path

Cause: Python interprets backslashes in Windows paths (\U, \D, \J) as Unicode escape sequences, causing the file path to be invalid.

Fix:

```
# WRONG
df = pd.read_csv("C:\Users\JUDITH\Downloads\file.csv")

# CORRECT — add r before the string (raw string)
df = pd.read_csv(r"C:\Users\JUDITH\Downloads\file.csv")
```

Reference:Link: [Stackoverflow]

Error 2: SyntaxError — stray parenthesis or double closing bracket

Cause: A stray opening (before a # comment, or an extra closing) at the end of a print() call, breaks the cell.

Fix:

```
# WRONG — stray ( before comment
df['FLAG_negative_qty'] = df['quantity'] < 0 (# domain check

# CORRECT
df['FLAG_negative_qty'] = df['quantity'] < 0  # domain check

# WRONG — double ))
print(f"  '{col}': {bad_dates.sum()} unparseable dates"))

# CORRECT — single )
print(f"  '{col}': {bad_dates.sum()} unparseable dates")
```

Error 3: KeyError — space inside a FLAG_ column name

Cause: A space was accidentally typed inside the column name string, e.g. 'FLAG_negative_ qty' instead of 'FLAG_negative_qty'.

Fix:

```
# WRONG — space between _ and qty
df['FLAG_negative_ qty'] = df['quantity'] < 0

# CORRECT — no space
df['FLAG_negative_qty'] = df['quantity'] < 0
```

Error 4: FLAG_ columns appear in outlier detection

Cause: Boolean FLAG_ columns are stored as integer 0/1 and are picked up by select_dtypes if cells are run out of order.

Fix:

```
# The filter below excludes FLAG_ columns
# BUT only works if cells are run top-to-bottom
numeric_cols = [c for c in numeric_cols
                if not c.startswith('FLAG_')]

# Best practice: Kernel -> Restart & Run All
```

Error 5: Date or ID checks produce no output

Cause: The date_cols or id_cols lists use different column names than the actual dataset. Column names are case-sensitive.

Fix:

```
# Step 1: print your actual column names
print(df.columns.tolist())

# Step 2: update lists to match exactly
date_cols = ["Order Date", "Ship Date"]   # example correction
id_cols   = ["Order ID", "Customer ID"]   # example correction
```

Reference: https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.columns.html

8 References

Official Documentation

Resource	Link
pandas documentation	https://pandas.pydata.org/docs/
pandas isnull()	https://pandas.pydata.org/docs/reference/api/pandas.isnull.html
pandas duplicated()	https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.duplicated.html
pandas to_datetime()	https://pandas.pydata.org/docs/reference/api/pandas.to_datetime.html
numpy documentation	https://numpy.org/doc/stable/
seaborn heatmap	https://seaborn.pydata.org/generated/seaborn.heatmap.html
Jupyter Notebook quickstart	https://jupyter-notebook.readthedocs.io/en/stable/notebook.html
Python raw strings	https://docs.python.org/3/reference/lexical_analysis.html